Adaptive Signal Processing and Machine Intelligence Coursework Part I

Han WU
CID: 01224459

A below average report. There is potential, however points are often superficially explained, showing a lack of insights and deep understanding. Some sections are left incomplete, and others are full with unnecessary graphs and tables, which distract the reader. The double column format makes it overall harder to read and comment.

Contents

Table of contents not aligned

Part I Spectrum Estimation **3**

1.1 DFT Basics 3

1.2 Properties of PSD 4

1.3 Resolution and Leakage of Periodogram-based Methods 9

1.4 Periodogram-based Methods Applied to Real–World Data 13

Part II Parametric and Line Spectra **15**

2.1 Correlation Estimation 15

2.2 Spectrum of Autoregressive Processes 18

2.3 Principal Component Analysis 19

2.4 Real World Signals: Respiratory Sinus Arrhythmia from RRI 21

Part III Adaptive signal processing **22**

3.1 The Least Mean Square (LMS) Algorithm 22

3.2 Adaptive Step Sizes 24

3.3 Adaptive Noise Cancellation 26

Part IV Widely Linear Filtering and Adaptive Spectrum Estimation **26**

4.1 Complex LMS and Widely Linear Modelling 26

References **28**

Adaptive Signal Processing and Machine Intelligence Coursework Part I

Double columns were used, for the nature of this report a single column would have been better.

Part I Spectrum Estimation

1.1 DFT Basics

a) Ideal spectrum and windowing

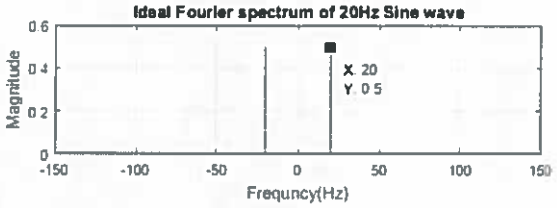


Figure 1: Ideal Fourier spectrum of 20Hz Sine wave

The ideal Fourier spectrum of a $20Hz$ sine wave is showed in Figure 1, which are two impulses at -20 and $20Hz$, with the magnitude of 0.5. This fact could be also verified via calculation.

$$\mathcal{F}_x[\sin(2\pi \times f0 \times x)]$$
$$= \int_{-\infty}^{\infty} e^{-2\pi i f0\, x} \left(\frac{e^{2\pi i f0\, x} - e^{-2\pi i f0\, x}}{2i} \right) dx$$
$$= \frac{1}{2} i\, [\delta(f + f0) - \delta(f - f0)] \qquad (1.1)$$

With sampling frequency $1kHz$, the DTFT of the 20Hz sine wave is illustrated in Figure 2. Notice that DTFT has period of 2π, which is the normalized frequency. However, in Figure 2 and Figure 3, the x-axis represents the real frequency, which means the period is $1000Hz$.

After applying an 20Hz rectangular window

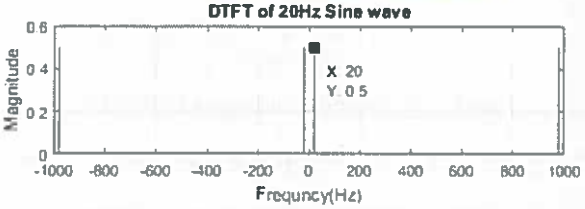


Figure 2: DTFT of original 20Hz sine wave

to the signal, the DTFT of original signal altered to Figure 3.

It shows that, for the rectangular window did truncation to the original signal, some high frequency components were introduced and side lobes would appear and the frequency drifted. This could be improved by increasing of window size N, for the size of mainlobe will decrease and provide higher frequency resolution.

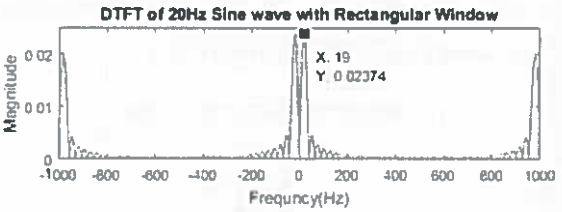


Figure 3: DTFT of 20Hz sine wave after windowing

b) Point number effects in DFT

The DFT of $20Hz$ sine wave was similar to the ideal Fourier transform and DTFT in the former section. Compared with Figure 1 and 2, Figure 3 seemed to be sampled in frequency domain. The reason is that, for periodic signal, after DTFT, the signal

should be continuous, while DFT comprises only discrete components in both time and frequency domain.

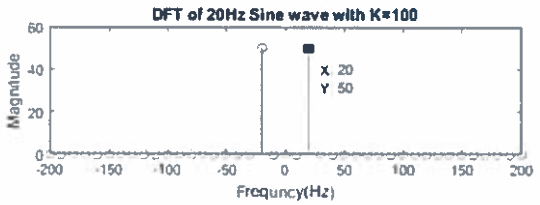


Figure 4: DFT of 20Hz Sine wave with K=100

Figure 5 shows 1000-point DFT of an $N = 100$ long sampled sine wave. Compared with Figure 4, points doing DFT is more than the number of sampled points. In this case, for the system will automatically padding zeros to the sampled signal, in order to make them have the same size, the frequency resolution of the original signal actually improved, frequency domain sampling is more intensive.

This manipulation is equivalent to applying a rectangular window in time domain, which could be verified through the similarity between Figure 5 and Figure 3.

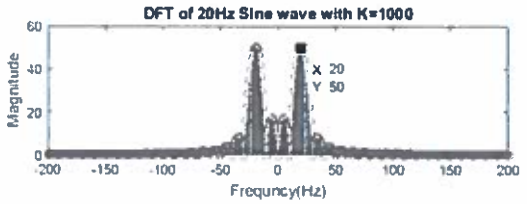


Figure 5: DFT of 20Hz Sine wave with K=1000

c) Incoherent sampling

With sampling frequency $f_s = 1000Hz$, there was no clear peak appears at 24Hz. This is due to incoherent sampling while doing DFT. In this case, the frequency resolution of a $K = 100$ point DFT is $\frac{f_s}{K} = 10Hz$. However, for $f = 24Hz$

$$\frac{f_R}{f_s} = \frac{f}{f_s/_K} = \frac{24 \times 100}{1000} = 2.4$$

Where f is the signal frequency, N is the number of sampled points. For the impulse at $f = 24Hz$ appears between the DFT bins $k = 2$ and $k = 3$, the DFT could not display accurate frequency of the signal.

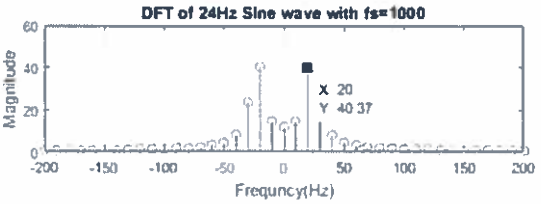


Figure 6: DFT of 24Hz Sine wave with fs=1000Hz

In order to achieve coherent sampling, the sampling frequency f_s was altered to $2400Hz$. For $f_s = 2400Hz$, the DFT bins would be located in Hz at $k\frac{f_s}{N} = 24 \times k$, where k is integer located in $[0, N]$.

Noticed that, the sampling frequency f_s can be any frequency that satisfied $f = k\frac{f_s}{N}$.

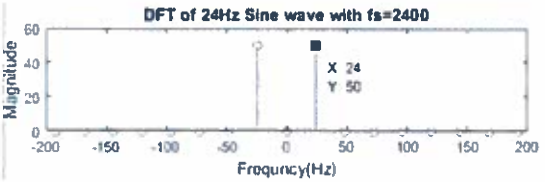


Figure 7: DFT of 24Hz Sine wave with fs=2400Hz

1.2 Properties of PSD

- **Analytically proof of PSD expression**

PSD represented by the average signal power over frequencies could be written as

$$P(\omega) = \lim_{N\to\infty} E\left\{\frac{1}{N}\left|\sum_{n=0}^{N-1} x(n)e^{-jn\omega}\right|^2\right\} \quad (1.2)$$

Expand the squared part in Eq. (1.2) into two sections:

$$P(\omega) = \lim_{N\to\infty}\frac{1}{N}\times E\left\{\sum_{k=0}^{N}\sum_{l=0}^{N} x(k)x(l)e^{-j2n\omega(k-l)}\right\} \quad (1.3)$$

$$= \lim_{N\to\infty}\frac{1}{N}\times \sum_{k=0}^{N}\sum_{l=0}^{N} E\{x(k)x(l)\}\, e^{-j2n\omega(k-l)} \quad (1.4)$$

For $E\{x(k)x(l)\} = r_{xx}(k-l)$, the PSD could be transformed into

$$P(\omega) = \lim_{N\to\infty}\frac{1}{N}\times \sum_{k=0}^{N}\sum_{l=0}^{N} g(k-l) \quad (1.5)$$

Where $g(k-l)$ represents the Fourier transform of autocorrelation. According to the lecture note, after doing IFT, the former equation could be written as

$$P(\omega) = \lim_{N\to\infty}\sum_{\tau=0}^{2N}\left(\frac{N-|\tau|}{N}\right) r_{xx}(\tau)\, e^{-j2n\omega\tau} \quad (1.6)$$

$$= \lim_{N\to\infty}\sum_{\tau=-(N-1)}^{N-1} r_{xx}(\tau)\, e^{-j2n\omega\tau} \quad (1.7)$$

$$-\lim_{N\to\infty}\sum_{\tau=-(N-1)}^{N-1}\frac{1}{N}\times|\tau| r_{xx}(\tau)\, e^{-j2n\omega\tau} \quad (1.8)$$

Where $\frac{N-|\tau|}{N} = 1-\frac{|\tau|}{N}$ could be regarded as a triangular window. given that the covariance sequence $r_{xx}(\tau)$ decays rapidly, which is

$$\lim_{N\to\infty}\frac{1}{N}\sum_{\tau=-(N-1)}^{N-1}|k| r_{xx}(k)\, e^{-j2n\omega k} = 0 \quad (1.9)$$

By substituting Eq. (1.9), Eq. (1.8) could be simplified as

$$P(\omega) = \lim_{N\to\infty}\sum_{\tau=-(N-1)}^{N-1} r_{xx}(\tau)\, e^{-j2n\omega\tau} \quad (1.10)$$

$$= \sum_{=-\infty}^{\infty} r_{xx}(\tau)\, e^{-j2n\omega\tau} \quad (1.11)$$

The result calculated above is exactly the same as the PSD defined using DTFT of the ACF which is

$$P(\omega) = \sum_{=-\infty}^{\infty} r_{xx}(\tau)\, e^{-j2n\omega\tau} \quad (1.12)$$

- **Simulations of two PSD expression**

After the analytical prove, a MATLAB simulation was conducted. The original signal applied was a squared wave with length $N = 10000$ Signal and its auto-correlation are demonstrated in Figure 8. As N turns to infinite, the autocorrelation turns to 0. According to Eq. (1.9), it shows that the covariance sequence $r(k)$ decays rapidly in this case.

poor academic english (poor english in general)

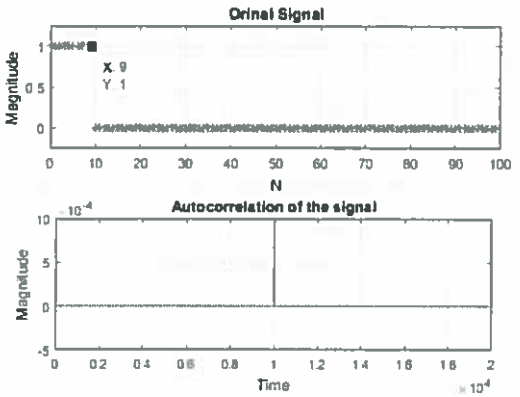


Figure 8: The original signal and its autocorrelation (rapidly decay r(k))

As Figure 9 illustrated, with fast decayed autocorrelation function, the two definition of PSD shows similar results.

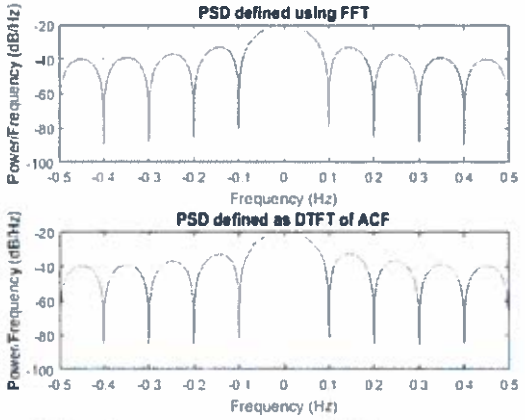


Figure 9: Two PSD based with rapidly decay r(k)

Then a sine wave was set as the original signal. Signal and its autocorrelation are demonstrated in Figure 10. In this case the covariance sequence $r(k)$ does not decay rapidly. Therefore, the result of two PSD generation methods had different results as Figure 11 shows.

figures are not saved in eps format

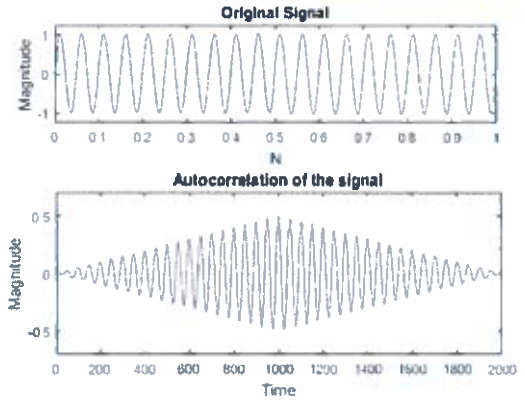


Figure 10: The original signal and its autocorrelation (without rapidly decay r(k))

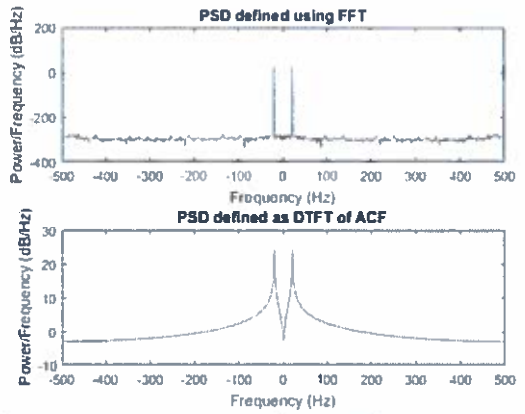


Figure 11: Two PSD based without rapidly decay r(k)

- **Simulations of fft () and p(ω)**

In order to prove that the method using DTFT to determine PSD from ACF gives the same performance of doing PSD directly, three signals were simulated. The first one is the direct FFT diagram without zero-padding in the middle. The bottom two plots were generated with zero-padded signal **x**. Second plot in Figure 12 is the DTFT of auto covariance function (ACF) after zero-padding with $r(k)$ has M = 10. Third plot represents the PSD generated by definition based on Eq. (1.2).

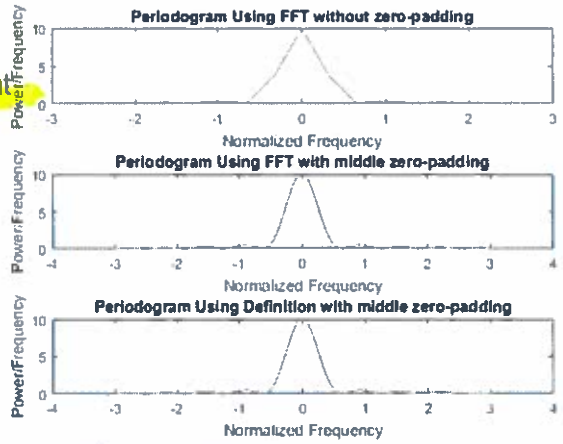


Figure 12: Performance of the command fft and PSD

As plot 2 and plot 3 shows that the method using DTFT to determine PSD from ACF gives the same performance of doing PSD by definition directly, which was also proved in the former section.

a) Zero padding properties of DTFT

(In this section, spectrums without fftshift would be illustrated.)

For $M = 10$, the size of auto covariance

function has size $2 \times M - 1 = 19$, in order to form vector **x** with length $L = 256$, 237 zeros were padded in the middle. From Figure 13, it shows that by using zero padding, as more frequency components were involved, periodogram became more smooth for the resolution of frequency improved as more components in **x** were involved.

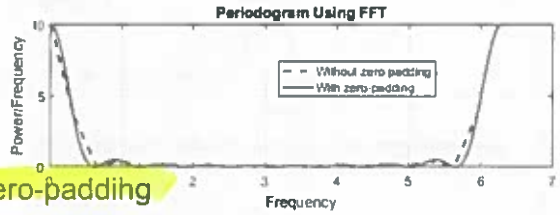


Figure 13: Periodogram with and without zero-padding when M = 10, L = 256

Effect of zero-padding were not fully understood/explored

However, for $M = 128$, the size of auto covariance function has size $2 \times M - 1 = 255$, in order to form vector **x** with length $L = 256$, only one zero was padded. The periodogram performance of padded and non-padded were same.

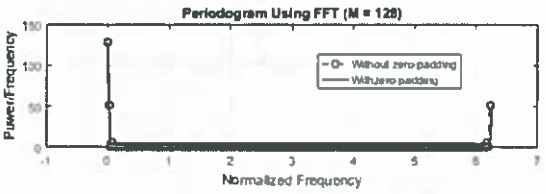


Figure 14: Periodogram with and without zero-padding when M = 128 , L = 256

This relies on the zero padding property of the DTFT and the property after inserting zeros. When the size of autocovariance function is similar to the size of the particular chosen **x** (**x** has size L, which is also the number of doing DTFT). However, when the size of ACF is much smaller than the number doing DTFT, the periodogram will be more smooth compared with the original one which are not padded.

b) Removing the imaginary part of x

Figure 15 and 16 illustrate the periodogram when the imaginary part of the spectrum is removed by replacing `xf` by `real(xf)` in both M=10 and M=128 cases.

It shows that by using the zero padding property of DTFT, the imaginary part actually contains few information of the original single, which means even the imaginary part of the spectrum is removed, the periodogram is still capable to provide correct result.

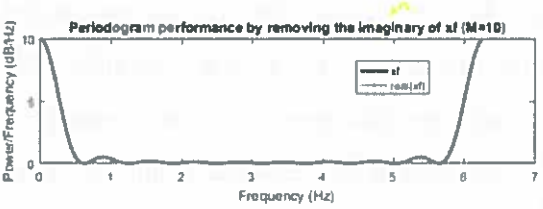


Figure 15: Periodogram performance by removing the imaginary of xf (M=10)

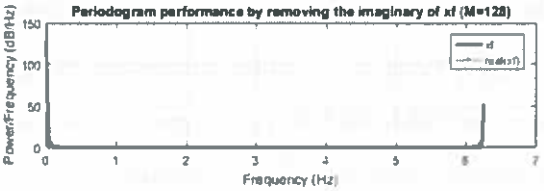


Figure 16: Periodogram performance by removing the imaginary of xf (M=128)

c) Another padding of x

Another zero padding method was applied in this section. Different from part b), in both M=10 and M=128 cases, by removing the imaginary part, erroneous periodogram appears.

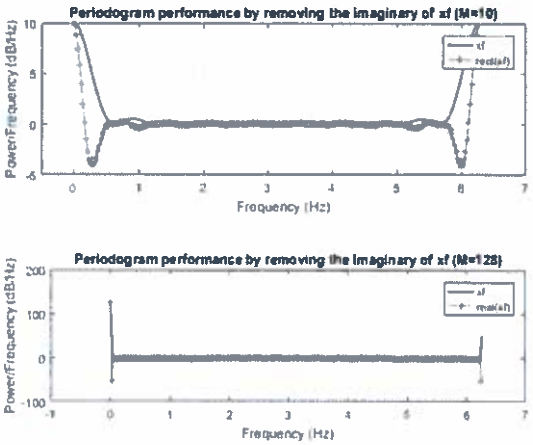


Figure 17: New padding method Periodogram performance after removing the imaginary of xf

In Figure 17, the red lines are the periodogram after removing the imaginary of `xf`, which are `real(xf)` while the blue lines are the original periodogram of `xf`.

The reason is that, by using this padding method, the imaginary components due to the round-off error is not negligible. The imaginary part also stores information of power, which should not be simply ignored.

d) Apply fftshift

In order to move the spectral estimates in the range $[0,2\pi]$ to a more convenient range $[-\pi,\pi]$, both fftshift( ) command and proper frequency vector to plot are required.

To ensure the shift command gives the spectral values at correct frequencies, the frequency vector `w` could be represented as

```
N  = length(xf);
w = (-N/2:N/2-1)/N*2*pi;
```

The frequency shifted version of the former parts are illustrated in Figure 18-20.

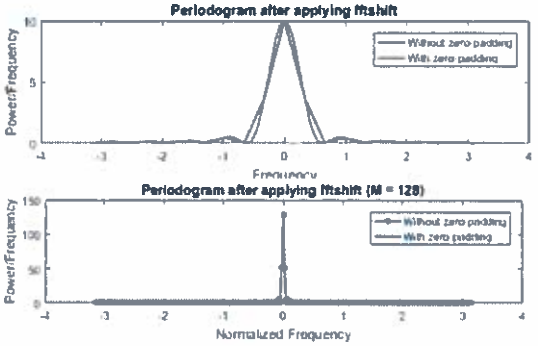


Figure 18: fftshift results of 1.2. a)

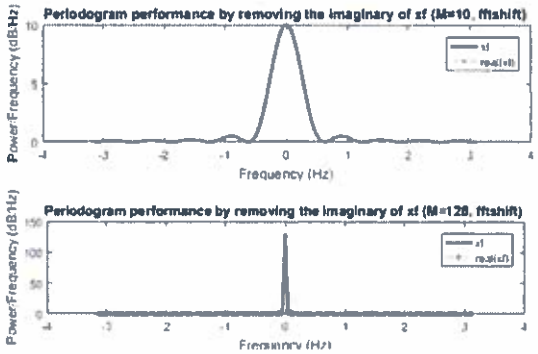


Figure 19: fftshift results of 1.2. b)

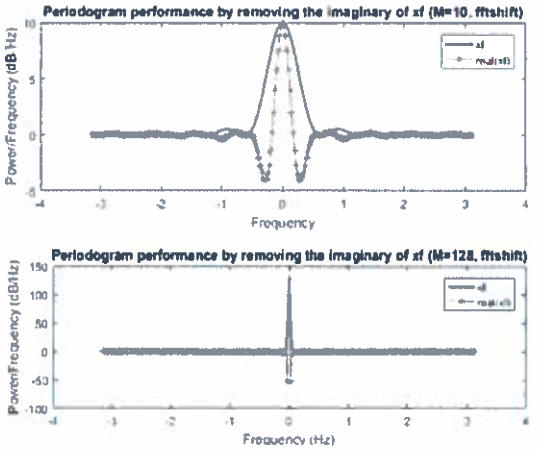


Figure 20: fftshift results of 1.2. c)

Explanations are often vague with no insight, showing in some cases superficial understanding.

No distinction between signals of even or odd lengths has been made.

1.3 Resolution and Leakage of Periodogram-based Methods

a) Bartlett window properties

According to Figure 22, it shows that the 3dB width of the main lobe of the Bartlett window $W_B(\omega)$ is proportional to 1/N, where N is the length of the Bartlett window.

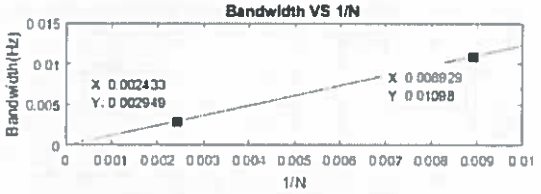


Figure 21: Bandwidth vs 1/N

However, the slop of the line showing above is

$$k = \frac{0.011 - 0.0029}{0.0089 - 0.0024} \approx 1.25$$

While for Bartlett window [1], 3dB width is supposed to be $1.28(\frac{2\pi}{N})$ (in Hz), thus k should be 1.28 theoretically.

One of the considering reason is that the measurement of 3dB width was not accurate. For there are no precise point generated at -3dB, the bandwidth was actually peaked by the system was the nearest point to the half-power.

Figure 23 illustrates the peaks of sidelobes (in dB) as a function of N.

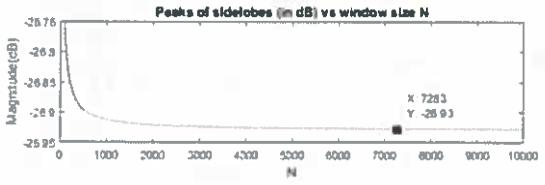


Figure 22: Side lobe level vs N

The simulation was conducted with different size of Bartlett window, which varies from 100 to 10000. With the increasing of window size N, the sidelobe level turns to be -27dB, which matches the theoretical value.

The previous two conclusions were then verified through plot of the magnitude of $W_B(\omega)$ for several values of N. Two examples of plotting are showed in Figure 23 and 24. Both linear and dB scale were considered in this case.

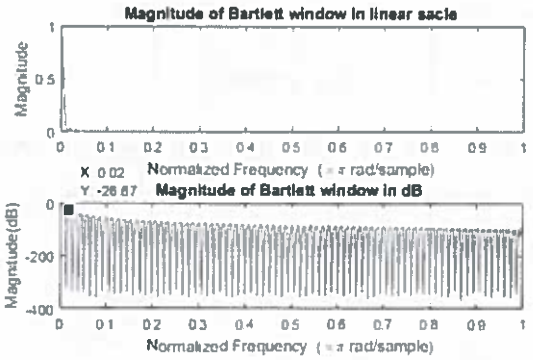


Figure 23: Bartlett window in linear and dB scale with N=300

graphs are not clear

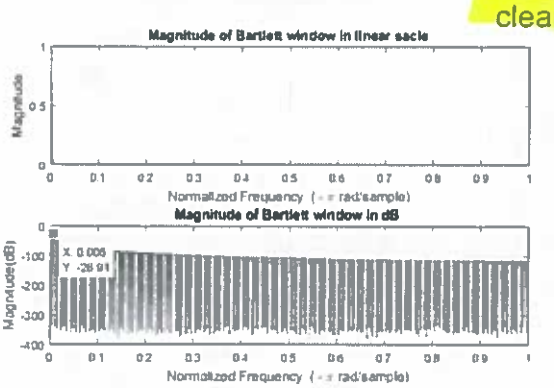


Figure 24: Bartlett window in linear and dB scale with N=1000

Therefore, it could be derived that the 3dB bandwidth of Bartlett window is proportional to 1/N, and the sidelobe level is approximately -27dB.

b) Bartlett-windowed Periodogram

Signal considered in this section is

$$x(n) = a1\sin(f \times 2\pi n + \phi_1) + a2sin((f + \alpha/N) \times 2\pi n + \phi_2) + w; \quad (1.13)$$

In order to resolve the frequency components, α need to be carefully selected to make sure that the difference between two frequency components is larger than the resolution of the periodogram. According to the lecture note, the resolution of periodogram for Bartlett window in

$$\Delta\omega \sim 0.89\frac{2\pi}{N} \quad (1.14)$$

Where N is the data length. This was proved by through simulation results demonstrated in Figure 25.

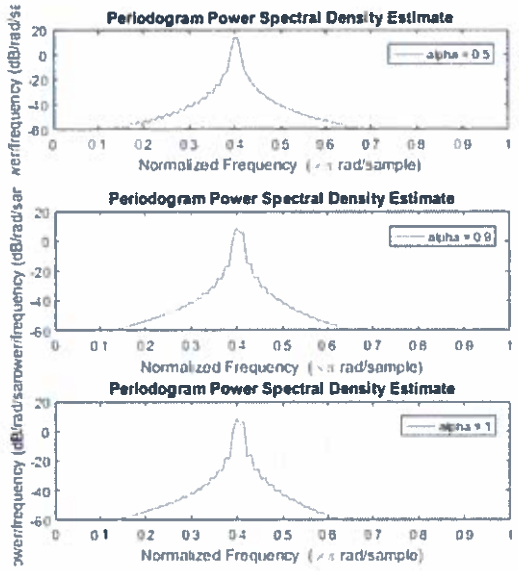


Figure 25: Periodogram of a = 0.5, 0.9, 1

When $\alpha = 0.9$, only one peak could be seen. With the increasing of α, it shows that when $\alpha = 0.9$, two frequency components could be identified. The separation between two signal components was $0.9 \times \frac{2\pi}{N}$, which is slightly larger than the resolution. By continue increasing the value of α, it becomes easier to distinguish two sine signals with different frequencies.

These three graphs could have been plotted on the same figure, to save space.

c) Hamming-windowed Periodogram

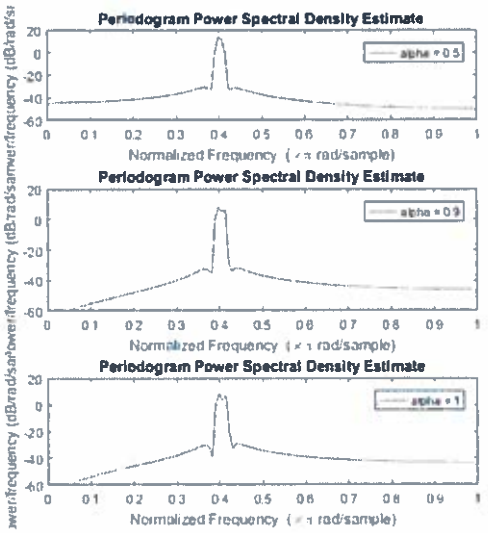


Figure 26: Periodogram of a = 0.5, 0.9, 1.1 (hamming window)

The performance of hamming windowed Periodogram was very similar to the Bartlett-windowed ones. The resolution threshold α is also around 0.9.

Compared with the pervious winnowed plot, the it shows that the sidelobes were dramatically decayed, especially the largest one. This is due to the property of hamming window that it could reduce the sidelobe level. This could be proved in Figure 27.

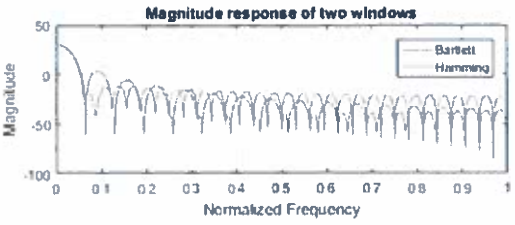


Figure 27: Magnitude response of two windows

d) Rectangular window and leakage

If the two sinusoidal components were well separated in frequency but differs in amplitudes. The high level sidelobe may also lead to problem of resolving two sine components.

When $\alpha = 4$, by using a rectangular window, the performance of periodogram with different amplitude of sinusoidal signal are showed in Figure 28.

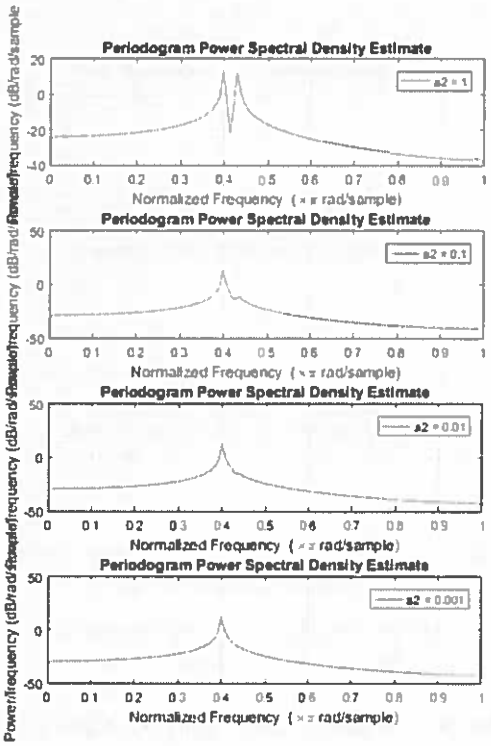


Figure 28: Periodogram PSD estimation with a2 = 1, 0.1, 0.01, 0.001 ($\alpha = 4$)

When the amplitude of the second sine component is small, it is hard to do the identification. As the figure demonstrated, it is possible to recognize the second sinusoidal term only when $a2 > 0.1$.

Another simulation was conducted with α altered to 12. In this case, the amplitude threshold $a2$ for identification also changes to 0.01.

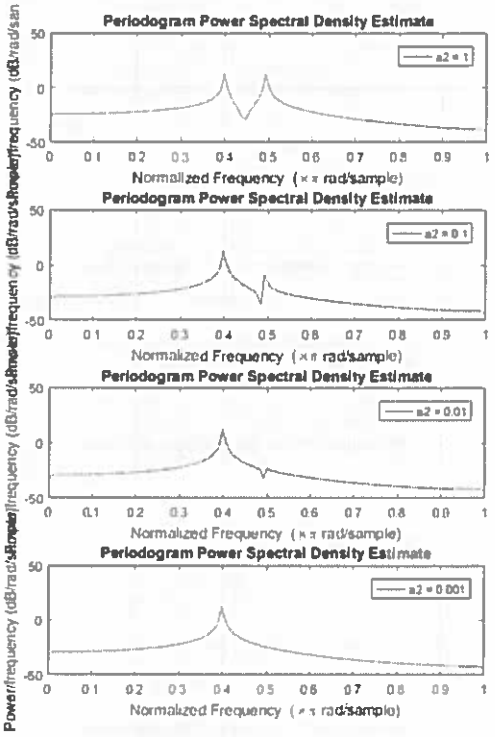


Figure 29: Periodogram PSD estimation with a2 = 1, 0.1, 0.01, 0.001 ($\alpha = 12$)

e) Bartlett window and leakage

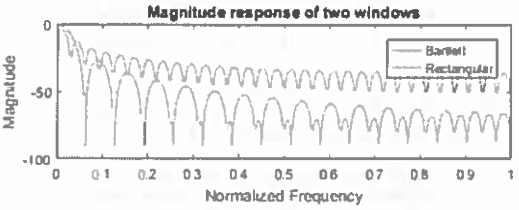


Figure 30: Magnitude response of two windows

Doing fourier transform for rectangular and Bartlett window, the magnitude response in dB is demonstrated in Figure 30. Compared with rectangular window, Bartlett window has lager mainlobe and lower sidelobe level, which means that the effect of leakage in this case might be milder compared with part c).

The simulation results are showed below:

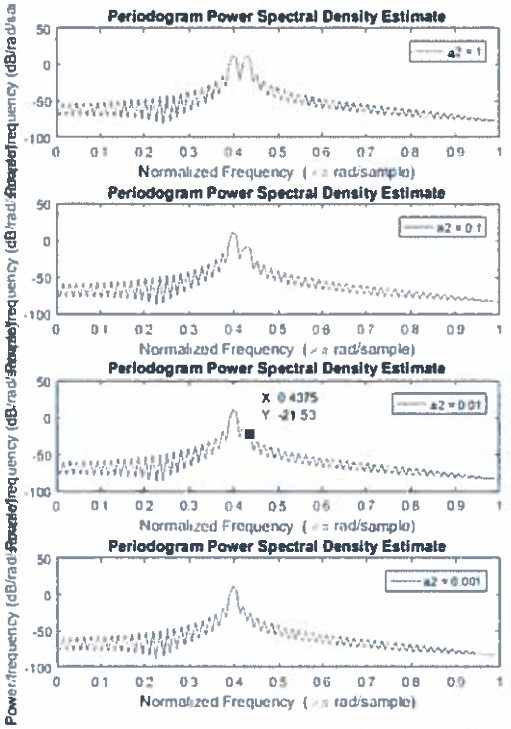


Figure 31: Periodogram PSD estimation with a2 = 1, 0.1, 0.01, 0.001 ($\alpha = 4$)

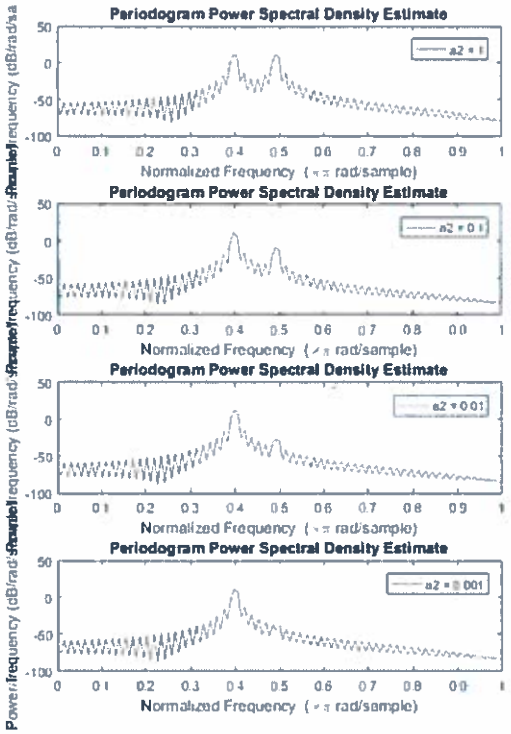


Figure 32: Periodogram PSD estimation with a2 = 1, 0.1, 0.01, 0.001 ($\alpha = 12$)

When $\alpha = 4$, by applying Bartlett window, now it is possible to recognize the second sinusoidal term only when $a2 > 0.01$ and when α changes to 12, the amplitude threshold now recuded down to 0.001. The simulation shows great improvement compared with rect-windowed cases.

f) Chebyshev window and Blackman-Tukey method

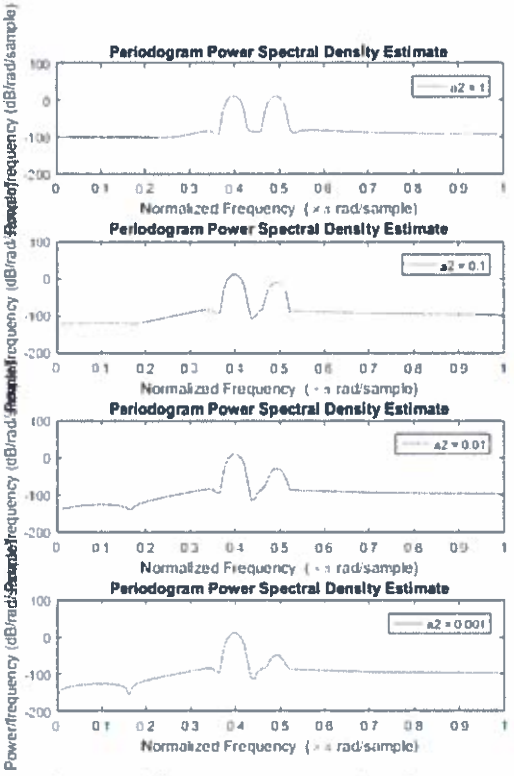


Figure 33: Periodogram PSD estimation with a2 = 1, 0.1, 0.01, 0.001 ($\alpha = 12$)

The periodogram after applying Chebyshev window is showed in Figure 33. In this case, it still provides satisfied amplitude threshold level. Noticed that the mainlobe is wider and it trades the reduction invariance for a reduction in the resolution.

The variance of the periodogram is decreased by reducing the variance of ACF Apply a window to ACF more rebost ACF will be

estimated over fewer data points. This is known as Blackman-Tukey method, and the periodogram will be 'smoothed' by using this method. The simulation results are displayed in Figure 34.

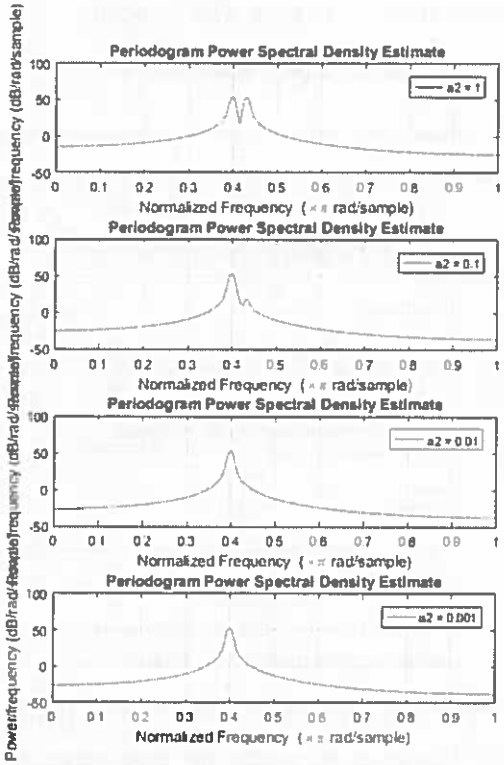


Figure 34: Periodogram PSD estimation with a2 = 1, 0.1, 0.01, 0.001 ($\alpha = 12$) using Blackman-Tukey method

It shows that, by using this method, though the periodogram was smoothed, the resolution decreased which increased the threshold of signal amplitude.

1.4 Periodogram-based Methods Applied to Real–World Data

a) Sunspot series processing

In this section, to avoid some errors may lead due to the automatic zero padding in command `periodogram()`, periodogram will be generated using definition, which was showed as Eq. (1.2).

The simulation result of doing periodogram based spectral estimation are showed in Figure 35.

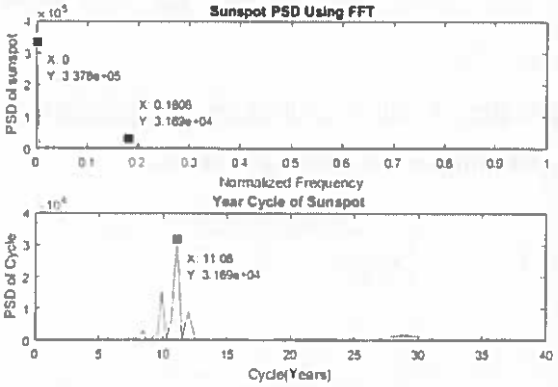


Figure 35: Periodogram and cycle plot of sunspots

After obtain the spectral estimation, the cycle of sunspot could be calculated based on the frequency, in which the peak appears in the periodogram. Thus the cycle is:

$$T = {}^{1}\Big/_{\frac{0.1806}{2}} = 11.08 \text{ years}$$

Which is also showed in Figure 35.

By removing the mean and trend from original data, the spectum was changed into Figure 36.

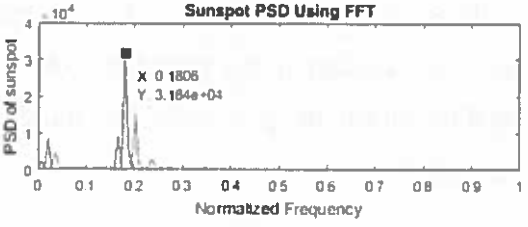


Figure 36: Periodogram after removing mean and trend

Both `mean` and `detrend` command in this part were used to remove the mean of the original signal, which is also the first

component after doing Fourier transform. In Figure 35, due to the effect of the mean components, the peak at $x = 0.1806$ is hard to recognize. After removing the first component in frequency domain, now the peak at the periodogram becomes more visible.

Figure 37 shows spectrum with logarithm was applied to the original data.

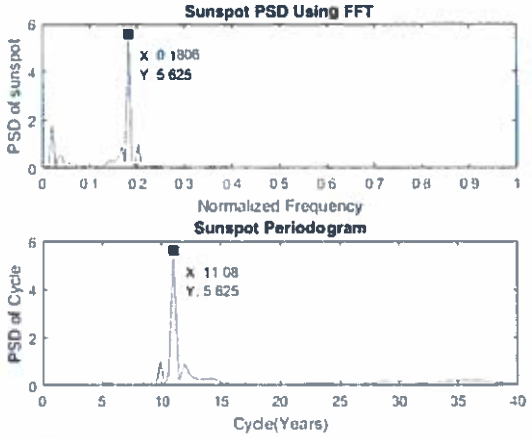


Figure 37: Periodogram after doing logarithm and removing mean and trend

By applying logarithm to the original signal, data smaller than 1 will be reduced, and only large components are preserved. Therefore, the performance of PSD is better compared with Figure 36. Though the magnitude, which is the y-axis, changes due to the logarithm calculation, the periodicity of the signal remains to the previous value, and the cycle is still 11.08 years.

b) EEG data processing

Simulations of applying averaged periodogram with different window lengths ($10\,s, 5\,s, 1\,s$) to the EEG data were showed in Figure 38. In both three cases, a peak at 13Hz could be identify in the spectrum within the range [11:20].

This spectrum average method is known as Bartlett's method. With the decreasing of window size L, variance was reduced.

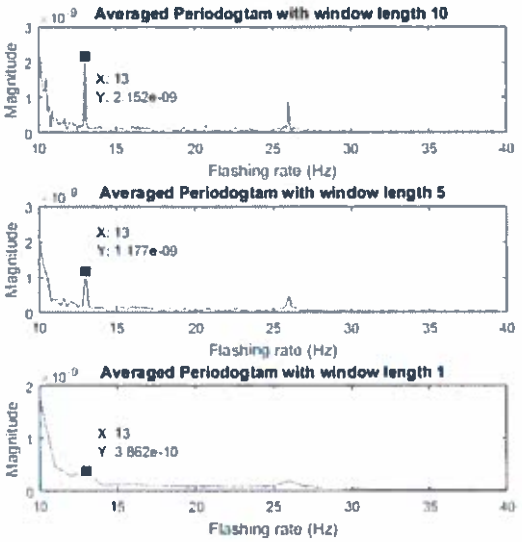


Figure 38: Averaged Periodogram with window length 10s, 5s and 1s

However, the spectral resolution was traded. From the plot, it could be seen that the mainlobe of spectrum became wider with the decreasing of window length.

To enable a fair comparison across all spectral analysis approaches, the number of frequency bins were adjusted to 5 DFT samples per Hz in both three cases.

The modified spectrum is showed in Figure 39. Now it is straightforward to distinguish the estimated SSVEP peaks from other spurious EEG activity in the surrounding spectrum compared with Figure 38.

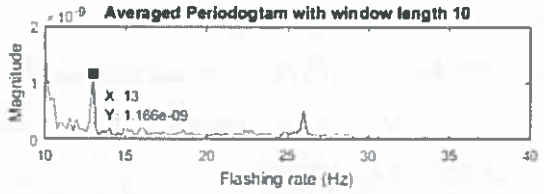


Figure 39: Averaged Periodogram with window length 10s under same frequency bins

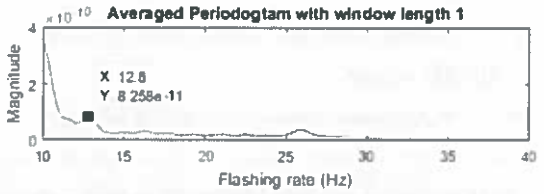


Figure 40: Averaged Periodogram with window length 1s under same frequency bins

With very small size of window size showed in Figure 40, the peak was still hard to recognize, for the resolution was reduced severely.

Part 1 was not bad overall, but it significantly took way too much unnecessary space, penalizing later sections.

Part II Parametric and Line Spectra

2.1 Correlation Estimation

a) Biased and unbiased ACF estimate

In Part I, it is proved that the PSD could be estimated based on correlation via:

$$P(\omega) = \sum_{k=-(N-1)}^{N-1} \hat{r}(k) e^{j\omega k} \qquad (2.1)$$

Where the autocorrelation function could be obtained using biased or unbiased estimator. For biased estimator, the autocorrelation is:

$$\hat{r}(k) = \frac{1}{N} \sum_{n=k+1}^{N} x(n)x^*(n-k) \qquad (2.2)$$

While the unbiased autocorrelation is:

$$\hat{r}(k) = \frac{1}{N-k} \sum_{n=k+1}^{N} x(n)x^*(n-k) \qquad (2.3)$$

Two simulations in biased and unbiased ACF were conducted, PSD estimation of both cases are illustrated in Figure 41 and 42.

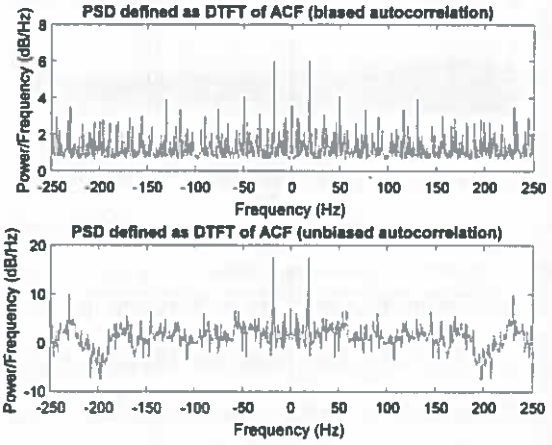


Figure 41: Biased and Unbiased based correlogram of WGN signals

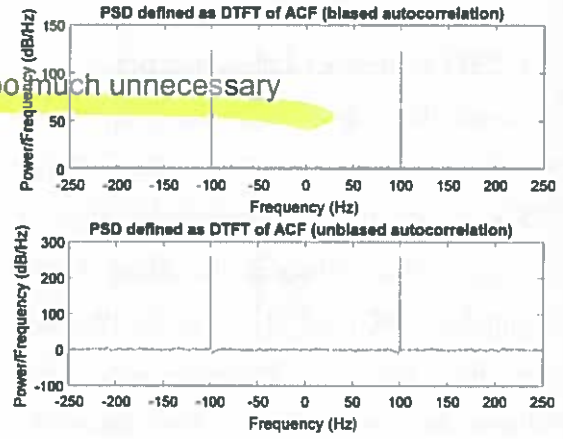


Figure 42: Biased and Unbiased based correlogram of noisy sinusoidal signals

It seems that the unbiased PSD estimation is more accurate its mean matches the true mean of PSD. However, the unbiased ACF estimate result in negative values when calculation PSD, which owing to the ACF may not be positive definite in this case.

The correlogram of biased and the unbiased ACF estimation are illustrated in the figure below. For large delays (i.e. when k close to N), it could be seen that the correlation

decrease slowly at first, but became dramatically large at the end of the plot.

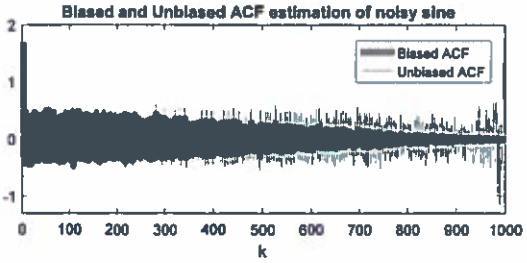


Figure 43: Biased and Unbiased ACF estimation of noisy sine

Thus even though this estimation is more accurate compared with the biased case, it may be highly erratic when large lags k was involved, for fewer samples are available when estimation PSD.

b) PSD estimate of random process

It is said that, when several realizations of a random signal are available, the estimate PSD could be represented by $\hat{P}(\omega) \pm \mu\hat{\sigma}_{P(\omega)}$, where $\hat{P}(\omega)$ is the mean of the estimated PSD and $\hat{\sigma}_{P(\omega)}$ is its standard deviation. Figure 44 illustrates an overlay plot of 100 generated PSD of two sinusoids signals immersed in WGN. The top plot represents mean while the standard deviation of the set shows in the bottom.

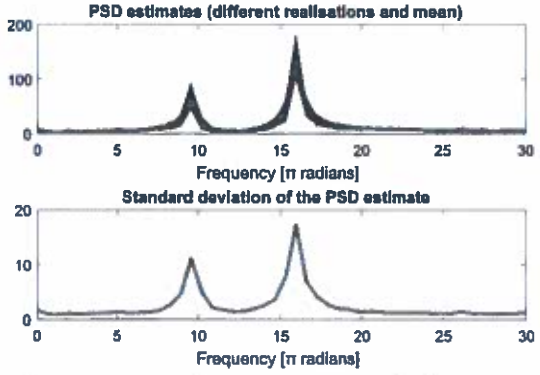


Figure 44: PSD estimate of two sinusoidal and WGN and its standard deviation

From the figure above, it shows that one of the drawback of this representation is that the standard deviation is proportional to PSD value, thus the confidence interval gets wider as PSD value increases.

c) PSD estimate of a random process in dB scale

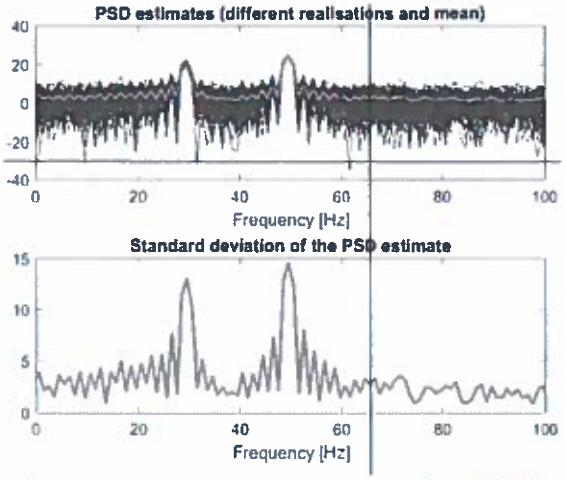


Figure 45: PSD estimate of two sinusoidal and WGN and its standard deviation (in dB)

By plotting the PSD estimates in decibels, it could be observed that, due to the contraction property of the logarithm, a more compact realization was achieved, which compensate the drawback in Figure 44.

d) Complex exponential signals PSD generation

PSD of signal, which consists of two complex sine waves with frequencies 0.3 Hz and 0.32Hz with additive complex white Gaussian noise is demonstrated in Figure 46. Observed that, from PSD, it was not able to identify the two frequencies of the signal. The reason is that the resolution of the periodogram being proportionate to $1/N$,

where N is the number of data samples. In this case, for the separation between the two frequencies is smaller than the resolution only one peak was showed in the figure.

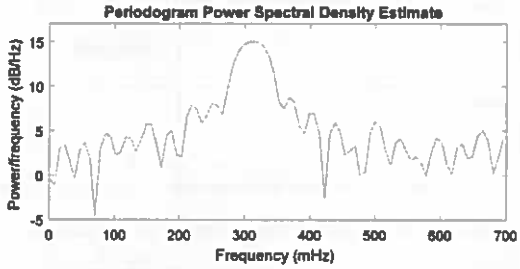


Figure 46: Periodogram of two complex exponentials with closely-spaced frequencies (n = 30)

PSD of two complex sine waves with frequencies 0.2 Hz and 0.32Hz with additive complex WGN is illustrated in Figure 47. Now the separation between the two frequencies is greater than the resolution, thus two peaks could be identified.

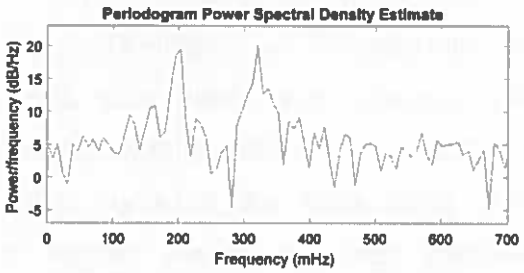


Figure 47: Periodogram of two complex exponentials with largely-spaced frequencies (n = 30)

Also, by considering more data samples, the periodogram starts showing correct spectra, which is showed in the following plot.

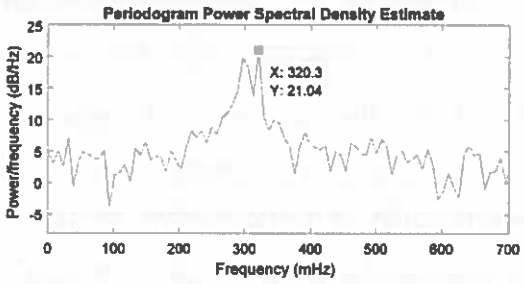


Figure 48: Periodogram of two complex exponentials with closely-spaced frequencies (n = 1000)

e) PSD generated by MUSIC method

The first line of MATLAB code in the instruction is `corrmtx` function. Function $[X, R] = corrmtx(x, M, METHOD)$, is used to calculate the autocorrelation matrix. Given a vector x (with length N) and an integer M, the function returns a matrix X.

In this case, $X'X$ is a biased autocorrelation estimation of S. The returned matrix X is constructed according to method *'mod'*, which represents *'modified'*, which corresponds to the modified covariance method. In some cases, this may able to generate more accurate estimates than covariance and the generated matrix X has dimension *2*(N-M)-by-M+1*. R is *a M+1 by M+1* matrix, which roughly equal to $X'X$.

Function `pmusic` is frequency estimation using the MUSIC eigenvector method, the function returns the S, which is the PSD of a signal whose correlation matrix estimated by the positive definite matrix R. P in the input represents the number of complex sinusoids in the signal. The input `'corr'` forces the input R to represent correlation matrix instead of matrix of signal. Sampling frequency was set to 1 Hz in this case. Output F is a vector of frequencies in Hz, which covers the range which PSD is calculated.

PSD estimation of two complex sine waves with frequencies 0.3 Hz and 0.32Hz with additive complex WGN using MUSIC is illustrated in Figure 49.

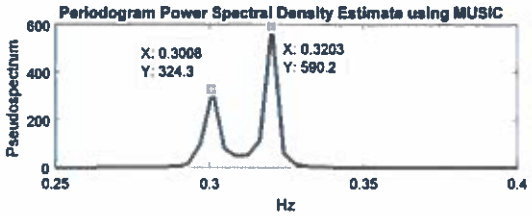


Figure 49: Periodogram of two exponentials with closely-spaced frequencies using MUSIC (n = 30)

It shows clearly, that the spectrum estimated using the MUSIC algorithm provide more detailed information.

The advantage of MUSIC is that, it is useful for frequency identification and it is also sensitive to model-order misspecification. However, though the performance of MUSIC is satisfied, this algorithm has a large cost in computation when searching over parameter space and also in storage.

2.2 Spectrum of Autoregressive Processes

a) Shortcomings of unbiased ACF

According to Eq. (2.3), it shows that although the unbiased ACF gives more accurate estimation compared with the biased case, it may be highly erratic when large lags k was involved. When calculating AR parameters, for the autocorrelation r_x is required, some errors may lead due to the unbiased ACF.

b) PSD of signal using model orders

Different orders were applied in PSD estimation. Some typical results are illustrated in the following figures.

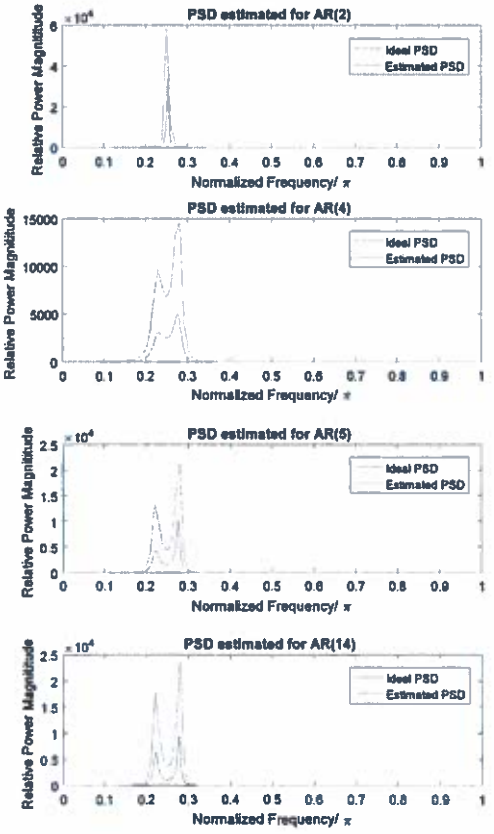


Figure 50: PSD estimation for p = 2,4,5,14

The order matters when doing the AR system PSD estimation. Appropriate order allows efficiently identification of peaks in the figure. If the model order is low ($p=2$), the frequency resolution is not enough to identify the peaks in the PSD diagram. When $p=4$, two peaks began to appear. E continuous increasing the order to 14, the peaks became more obvious. However, noticed that the order cannot be increased unlimited, for some 'false peak' may occur.

In each plot, the blue lines are the practical PSD while the red are estimated results. It is clear that both of the two sets of plot shows the same trend with the increasing of order.

c) PSD of signal using model orders using more data points

By increasing the data length to 10000, the simulation results are showed as:

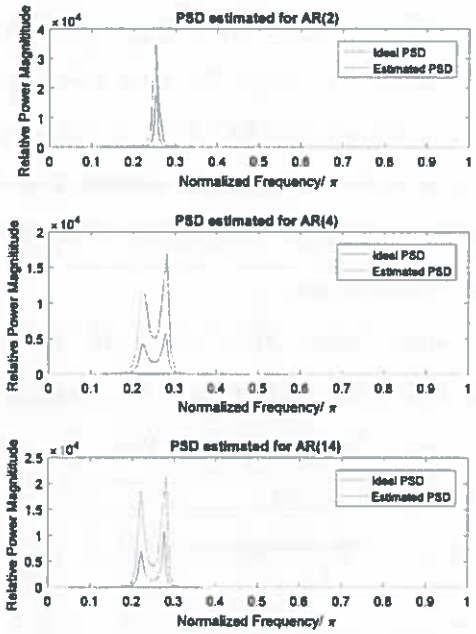


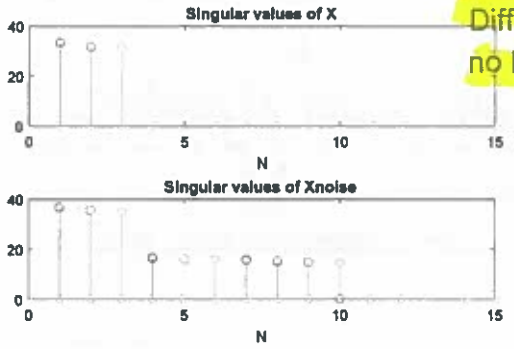
Figure 51: PSD estimation for p = 2,4,14

Similar to previous results, when the chosen model order is lower (under-modelling) than the correct AR(4) model order, PSD plot could not tell the two peaks of the signal, while for higher order(over-modelling), the resolution increases and makes it easier to identify. Traditional PSD estimation depends on the length of data samples, while this method does not rely on the sample numbers.

2.3 Principal Component Analysis

a) Singular value decomposition

After doing singular value decomposition to X and $Xnoise$, the singular values are illustrated in Figure 52. According to the definition, rank of matrix is equal to the number of non-zero elements in singular value. Thus the rank of **X** is 3.



Different colours, no legend

Figure 52: Singular values of X and Xnoise

The square error between each singular value of **X** and **Xnoise** is showed below.

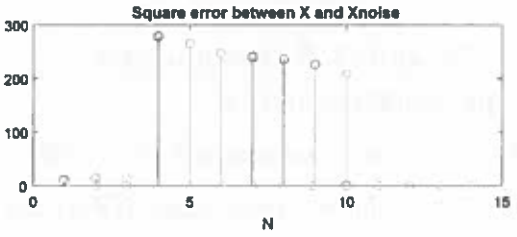


Figure 53: Square error between X and Xnoise

After the noise involved, the rank of signal altered from 3 to 10, which is the same as the rank of noise. It became hard to identify the rank of the matrix **Xnoise**, when the noise power is too high or low.

b) Low rank denoise approximation

Using only $r = 3$ most significant principal components, a low-rank approximation, denoised matrix of X_{noise} was created, which is called $\hat{X}_{noise}$.

Figure 54 shows a comparison of the difference between the variables (columns) of the noiseless input matrix X and denoised matrix $\hat{X}_{noise}$ and Figure 55 is the norm of the difference between X and denoised

$\hat{X}_{noise}$ in each column.

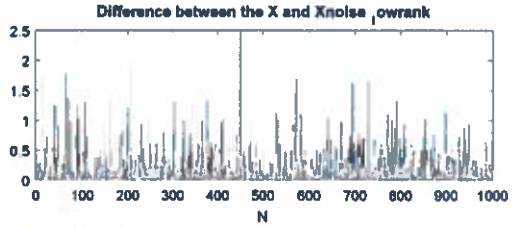


Figure 54: Difference between X and denoised Xnoise

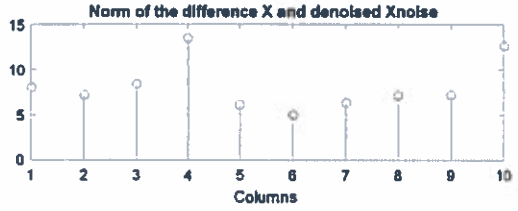


Figure 55: Norm of the difference between X and denoised Xnoise

c) OLS and PCR estimation for parameter matrix B

The output data are obtained by $\boldsymbol{Y} = \boldsymbol{XB} + \boldsymbol{N_Y}$. The ordinary least squares (OLS) and principal component regression (PCR) methods are applied to solve the unknown regression matrix B. Difference between $\boldsymbol{Y}$ and $\hat{\boldsymbol{Y}}_{OLS}$ and $\hat{\boldsymbol{Y}}_{PCR}$ are showed in Figure 56.

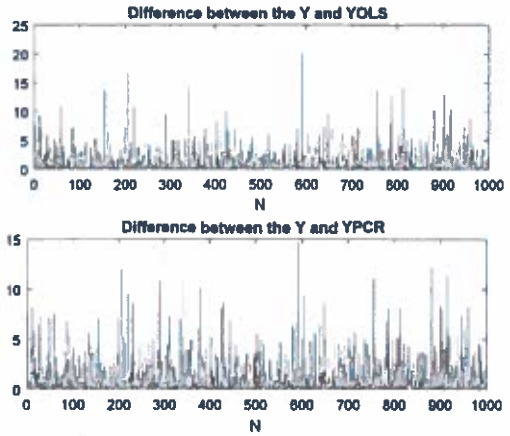


Figure 56: estimation error between Y and YOLS and YPCR

It seems that the bottom plot has lower error level. In order to be more concise, norm value between $\boldsymbol{Y}$ and $\hat{\boldsymbol{Y}}_{OLS}$ and $\hat{\boldsymbol{Y}}_{PCR}$ were calculated as well, which are separately 45.119 and 41.476. Thus the PCR method, which has lower level of error, provides better approximation, compared with OLS.

MATLAB was made a warning when doing the calculation of OLS. The OLS solution is given by $\boldsymbol{B_{OLS}} = (\boldsymbol{X^H X})^{-1}\boldsymbol{X^H Y}$. In this case, as $\boldsymbol{X}$ is sub-rank and the matrix $\boldsymbol{X^H X}$ is singular and so the calculation of $(\boldsymbol{X^H X})^{-1}$ becomes intractable.

Test data were also involved in the simulation. The performance by comparing $\boldsymbol{Y_{test}}$ and $\hat{\boldsymbol{Y}}_{test-OLS}$ and $\hat{\boldsymbol{Y}}_{test-PCR}$ are showed in Figure 57.

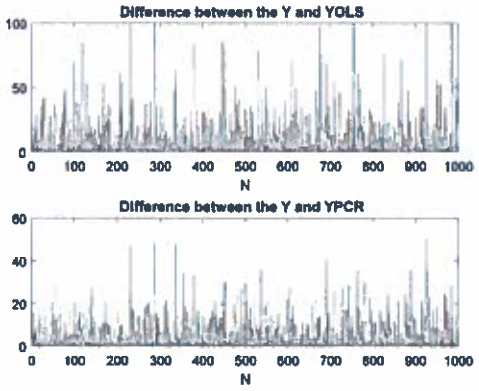


Figure 57 estimation error between Ytest and Ytest-OLS and Ytest-PCR

The norm value of OLS and PCR are 128.184 and 89.350, which satisfied the performance of the previous simulation.

d) Effectiveness of OLS and PCR

In order to assess the effectiveness of the PCR compared to the OLS solution, the estimated regressions coefficients, $\hat{\boldsymbol{B}}$ was tested over an ensemble of test data.

Using the same PCR and OLS regression

coefficients as in (c), mean square error (MSE) estimates for the PCR and OLS are 0.463 and 0.591, which means the PCR has better effectiveness compared with OLS method.

The peaks were not given in breaths per minute, as required by the assignment.

2.4 Real World Signals: Respiratory Sinus Arrhythmia from RRI

a) PSDs of the RRI data

Figure 58 shows the standard periodogram as well as the averaged periodogram with different window lengths (20 and 100s) to obtain the power spectral density of the RRI data of the first trial.

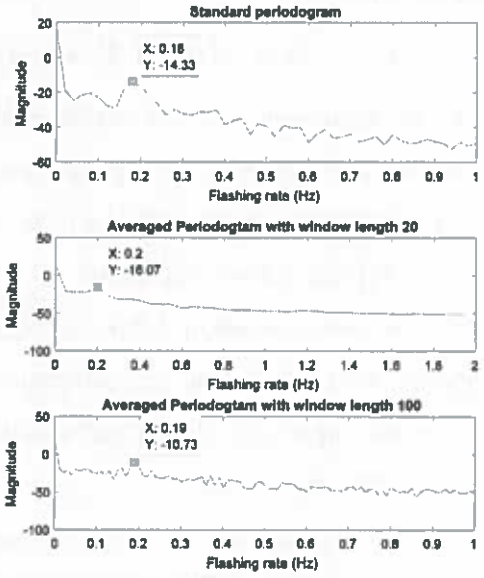
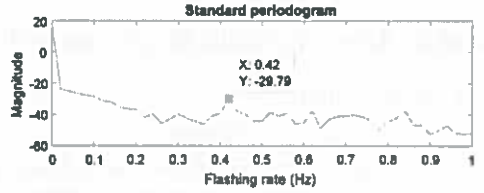


Figure 58: Standard and averaged periodogram in window length 20s and 100s of first trial

Same processes were applied to the second and third trial of the data, which are showed in Figure 59 and Figure 60 separately.



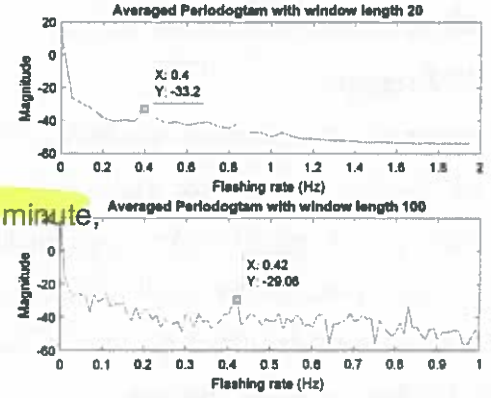


Figure 59: Standard and averaged periodogram in window length 20s and 100s of second trial

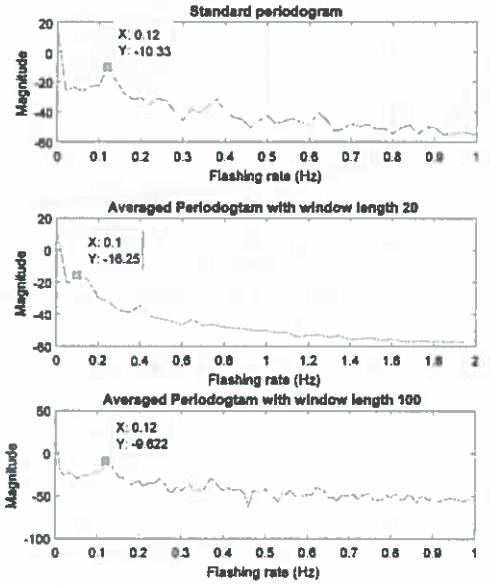


Figure 60: Standard and averaged periodogram in window length 20s and 100s of third trial

b) PSD analyse of the RRI data

Compared among Figure 58 to 60, it is obvious that peaks occur in different positions and for different window length, the resolution of the plot also changes. For three trials, peaks could be identified in the spectrum, which are 0.19Hz, 0.4Hz and 0.1Hz, separately. These three peaks correspond to frequencies of respiration for the three experiments.

c) AR spectrum estimation for the RRI signals

AR spectrum estimation was conducted for the RRI signals for the three trials. To find the optimal AR model order, experiment orders were started by 10 in all three cases and started increasing the order until peaks could be observed in the spectrum.

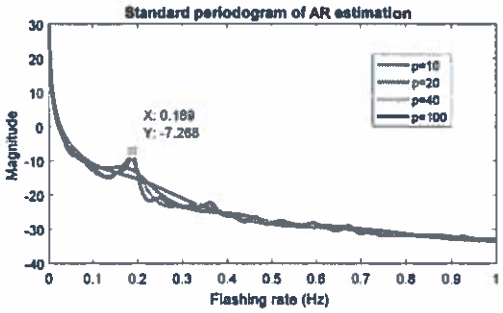


Figure 61: AR estimation with different order of the first trial

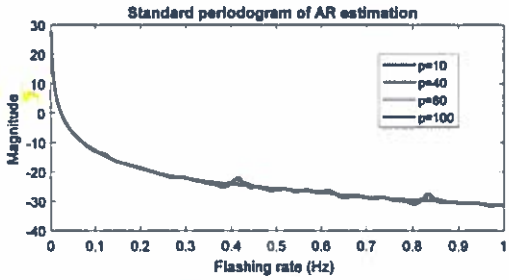


Figure 62: AR estimation with different order of the second trial

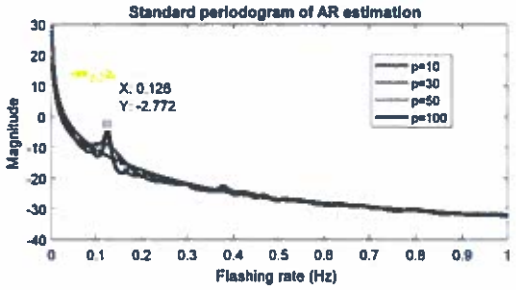


Figure 63: AR estimation with different order of the third trial

A proper order for first trial is 40, while 80 and 50 is more suitable for second and third trial separately. The peaks in Figure 61 to 63 are (approximately) corresponding to the theoretical respiration rate. Compared with periodogram estimate in Part a), the estimated AR spectrum is smoother and the unnecessary peaks were supressed.

Part III Adaptive signal processing

3.1 The Least Mean Square (LMS) Algorithm

a) The LMS filter setup

Signal $x(n)$ was set to a second-order auto-regressive process, which satisfies the difference equation

$$x(n) = a1x(n-1) + a2x(n-2) + \eta(n) \quad (3.1)$$

In order to implement an LMS estimator of the AR coefficients a1 and a2 using the signal x(n), firstly the input vector $\boldsymbol{x}(n) = [x(n-1);\ x(n-2)]$ and correlation matrix of the input vector need to be setup. After doing the calculation, it is found that, the correlation matrix of the input vector is a 2x2 matrix, which could be written as:

$$\begin{bmatrix} r(0) & r(1) \\ r(1) & r(0) \end{bmatrix}$$

mathematical derivation is completely missing.

Where r is the autocorrelation of the signal x. The step size μ, which is the key parameter which controls how the adaptive filter behaves should force the LMS converge in the mean. Thus it should be chosen to lie within in the range

$$0 < \mu < 1/LP_x \quad (3.2)$$

where P_x is the power of the input signal to

a specific value for mu is not given

the adaptive filter. Sometimes the step size is calculated by multiply the upper bound by 1/3 of Eq.(3.2).

b) Implementation of an LMS adaptive predictor

Squared prediction error in dB scale $10\,log(e^2(n))$, was estimated along time using two different step sizes $\mu = 0.05$ and $\mu = 0.01$. The learning curves, which averaging the plots of $10\,log(e^2(n))$ of 100 different realizations were illustrated in Figure 64 and 65.

Again, could have plotted these together to save space.

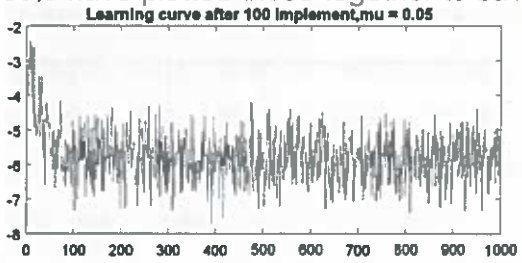


Figure 64: Learning curve after 100 implement, mu = 0.05

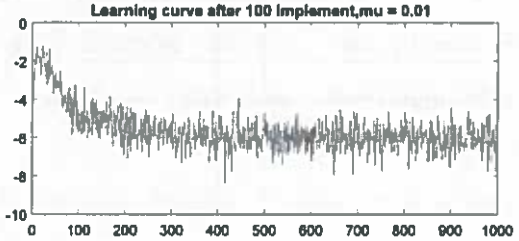


Figure 65: Learning curve after 100 implement, mu = 0.01

It shows that, though for larger step size, the system learns fast, the steady state error is quite large and not stable enough. Smaller step size provided better performance, though the learning speed was slow.

c) Misadjustment estimation

Theoretical LMS misadjustment is given by

$$\mathcal{M}_{LMS} \approx \frac{\mu}{2} Tr\{R\} \quad (3.3)$$

where $\boldsymbol{R}$ is the autocorrelation matrix of the input, μ is the step size and Tr denotes the matrix trace operator. While the estimated value is given by the ratio of EMSE (excess mean square error and the minimum mean square error.

$$MSE = \lim_{n\to\infty} \mathbb{E}\{e^2(n)\} = \sigma_\eta^2 + EMSE \quad (3.4)$$

$$\mathcal{M}_{LMS} = \frac{EMSE}{\sigma_\eta^2} \quad (3.5)$$

After the simulation, it is found that when $\mu = 0.05$ the practical misadjustment is 0.056 and the theoretical value is 0.051.

For $\mu = 0.01$, the estimated and theoretical misadjustment are 0.0074 and 0.084 separately.

d) Coefficient estimation

silly typo

To estimate the adaptive filter coefficients for the step-sizes of $\mu = 0.05$ and $\mu = 0.05$, 100 independent trials of the experiments were conducted. By averaging the steady-state values of the coefficients, the results are showed in the following table.

Table 1: True and estimated coefficients

good use of table

	a1	*a2*
True coefficients	0.1	0.8
$\mu = 0.05$	0.072	0.70
$\mu = 0.01$	0.093	0.776

Compare these estimated values with the true coefficients, smaller step size achieved better performance. There is a trade-off between the learning speed and the steady-state error.

e) Coefficient update of leaky LMS algorithm

not a right justification, showing poor understanding.

If the autocorrelation matrix of input signal *x(n)* has zero eigenvalues, by applying LMS algorithm, the final weights may not converge and lead to instability. By introducing a leakage coefficient, $0 < \gamma < 1$, filter weights could be forced to converge.

```
w = (1-mu*gamma)*w + mu*x*e(k);
```

See code in Part III for LMS coefficient update that $w(n)$ that minimizes the cost function.

f) Implementation of leaky LMS algorithm

a graph together with the tables could have helped

Different values for γ and μ were used to estimate the AR coefficients of the signal using the leaky LMS algorithm. Simulation results are showed in Table 2 and 3.

Table 2: Coefficient estimation when mu=0.01

$\mu = 0.01$	*a1*	*a2*
$\gamma = 0.0001$	0.0895	0.7699
$\gamma = 0.01$	0.1887	1.5282
$\gamma = 0.3$	0.3160	2.0703
$\gamma = 0.5$	0.4782	2.5414
$\gamma = 0.7$	0.6172	2.9500

Table 3: Coefficient estimation when mu=0.05

$\mu = 0.05$	*a1*	*a2*
$\gamma = 0.0001$	0.0765	0.7093
$\gamma = 0.01$	0.1388	1.4038
$\gamma = 0.3$	0.2304	1.8936
$\gamma = 0.5$	0.3371	2.2888
$\gamma = 0.7$	0.4354	2.6461

When γ is small, the simulation results were similar to LMS algorithm without leakage coefficient. With the increasing of γ, the weights of the leaky LMS algorithm converged to incorrect parameters a1 and a2.

The reason may be that in this case the autocorrelation matrix of *x(n)* does not have zero eigenvalues, thus this method compensated the instability that was not exist. Therefore, the stability of the simulation was, which causes the parameter not convergent.

3.2 Adaptive Step Sizes

a) Gradient adaptive step-size (GASS) algorithms

Step-size chosen for the LMS is important due to the trade-off between convergence speed and steady state error. In order to achieve better performance of the adaption, LMS algorithm with an updated step size $\mu(n)$ was introduced:

$$\boldsymbol{w}(n+1) = \boldsymbol{w}(n) + \mu(n)e(n)\boldsymbol{x}(n) \quad (3.6)$$

The learning rate $\mu(n)$ was adapted using GASS algorithms and $\mu(n)$ is updated according to

$$\mu(n+1) = \mu(n) + \rho e(n)\boldsymbol{x}^T(n)\psi(n) \quad (3.7)$$

Benveniste, Ang & Farhang and Matthews & Xie provide three method of updating $\psi(n)$. Weight error curves for each updating methods was simulated, the results were showed below.

Compared the performance with a standard LMS algorithm, with the same initial learning rate $\mu = 0.01$, the GASS methods converges faster which even comparable to

the curve of $\mu = 0.1$ with less steady state error. But the computational cost is higher than the fixed step size.

The different filter types were not sufficiently explored in terms of parameters used.

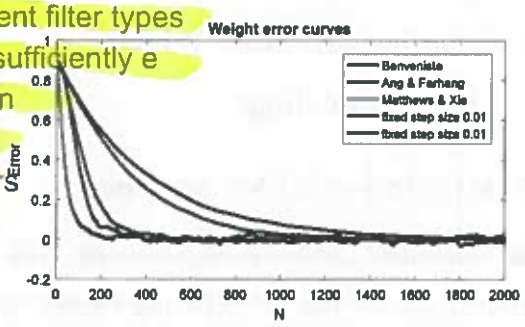


Figure 66: Weight error curves of GASS and fixed u

With the same initial converging speed, the Benveniste updating method converges fastest, while the Matthews converges slowest, which is close to the fixed step simulation result. Also compare with large μ, though the GASS method converges quickly, the error at the steady state is much smaller.

Thus by applying GASS method, the step-size could be increased at the start of the adaptation and getting smaller when approaching the ideal value, which means it achieves fast convergence and low steady state error at the same time.

b) Verify update equation

In original LMS algorithm, the error is estimated by

$$e(n) = d(n) - x^T(n)w(n) \quad (3.8)$$

A posteriori error is introduced

$$ep(n) = d(n) - x^T(n)w(n+1) \quad (3.9)$$

The weight updated equation is

$$w(n+1) = w(n) + \mu ep(n)x(n) \quad (3.10)$$

Substituting Eq.(3.9) into Eq.(3.10)

$$w(n+1) = w(n) + \mu(d(n) - x^T(n)w(n+1)) \times x(n) \quad (3.11)$$

Thus the difference after updating is

$$\Delta w(n) = w(n+1) - w(n) \quad (3.12)$$

$$= \mu(d(n) - x^T(n)w(n+1)) \times x(n) \quad (3.13)$$

Then substituting Eq. (3.8)

$$w(n+1) - w(n) = \mu(e(n) - x^T(n)w(n) - x^T(n)w(n+1)) \times x(n) \quad (3.14)$$

Thus

$$(1 + \mu\|x(n)\|^2)(w(n+1) - w(n)) = \mu e(n)x(n) \quad (3.15)$$

Therefore

$$\Delta w(n) = \mu e(n)\left[1 - \mu\frac{\|x(n)\|^2}{1 + \mu\|x(n)\|^2}\right]x(n) \quad (3.16)$$

As this method is equivalent to normalized LMS (NLMS) algorithm with form

$$w(n+1) = w(n) + \frac{\beta}{\epsilon + \|x(n)\|^2}e(n)x(n) \quad (3.17)$$

$$\frac{\beta}{\epsilon + \|x(n)\|^2} = \frac{\mu}{1 + \mu\|x(n)\|^2} \quad (3.18)$$

It can be derived that $\mu = 1/\epsilon$ and $\beta = 1$.

c) Generalized normalized gradient descent algorithm

By making the regularization factor ϵ time varying $\epsilon(n)$ in NLMS, the adaptive algorithm is now called GNGD algorithm. Implement the GNGD algorithm compared with Benveniste's algorithm is showed in Figure 67.

It shows that GNGD method provided faster convergence speed compared with the Benveniste's GASS algorithm, while the

steady state error variance was quite large, which is not satisfied.

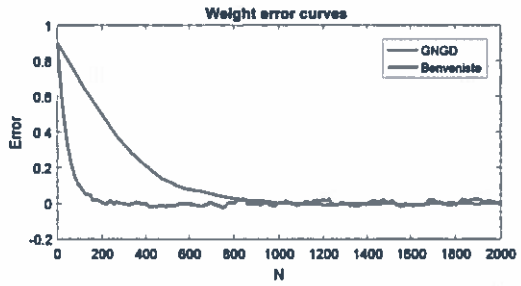


Figure 67: Weight error curves of GNGD and Benveniste's GASS algorithm

If not considering the operation in matrix, assume that there are M elements in input vector, thus for Benveniste's GASS algorithm, in order to obtain new weight, it requires 9 multi-plications and 4 additions every time and for GNGD method, 10 multiplications and 4 additions are required each time. Therefore, the computational complexity of GNGD is less than Benveniste's algorithm.

complexity analysis not satisfactory

3.3 Adaptive Noise Cancellation

this section is missing

a) Adaptive line enhancement (ALE) using the LMS algorithm

The Mean Square Error is

$$\mathbb{E}\left\{\left(s(n)-\hat{x}(n)\right)^2\right\} = \mathbb{E}\{(s(n)-\mathbf{w}^T(n)\boldsymbol{u}(n))^2\}$$

For delay Δ is chosen such that noise in the signal $s(n)$ and the predictor input $\boldsymbol{u}(n)$ are uncorrelated, by expanding the Mean Square Error it can be derived that Δ=2. MATLAB program for the adaptive line enhancement using the LMS algorithm is showed in Part III code.

Part IV Widely Linear Filtering and Adaptive Spectrum Estimation

4.1 Complex LMS and Widely Linear Modelling

a) ACLMS and CLMS implement

The learning curve, averaged after 100 simulations, for the ACLMS and CLMS is illustrated in Figure 68.

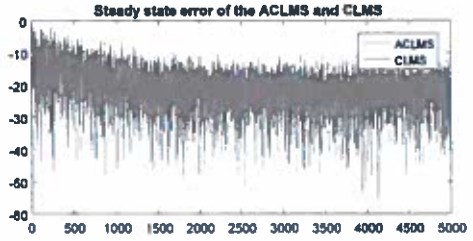


Figure 68: Learning curve of ACLMS and CLMS

Steady state error of the ACLMS and CLMS is similar, which is close to -20dB. There is a problem that even doing 100 times trial, the line was still not smooth enough, however the result of estimated coefficient is very close to the ideal value.

b) Wind data in reality

Figure below are the scatter diagram of three wind regimes. The circularity of the complex wind signal for medium and high wind speeds are high, while for low wind regime, the wind has bad circularity, for the plot seems more like an ellipse instead of a circle.

Comparison between ACLMS and CLMS should have shown that the augmented version is more effective in capturing higher order statistics.

Concept of circularity not understood. A plot of all the three regimes together could have helped.

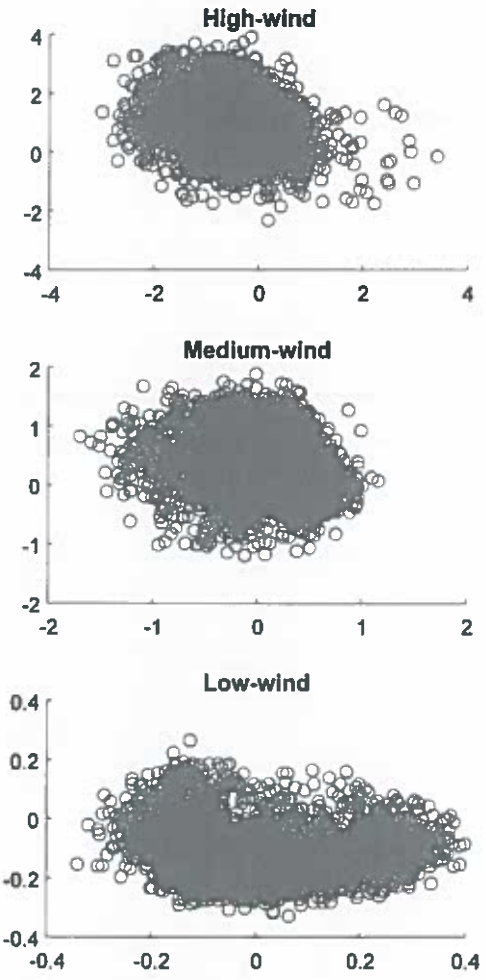
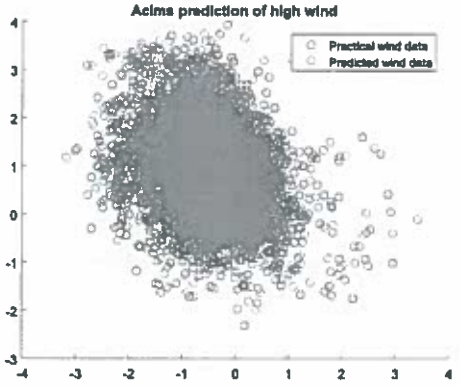


Figure 69: Scatter diagram of three wind regime

The prediction of three regimes with ACLMS and CLMS separately are showed below.



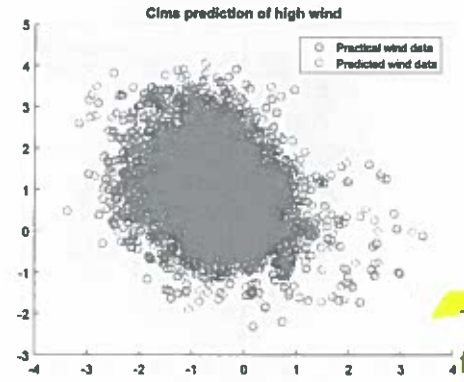


Figure 70: ACLMS and CLMS estimation in high wind

This page is full of unnecessary graphs: wasted space.

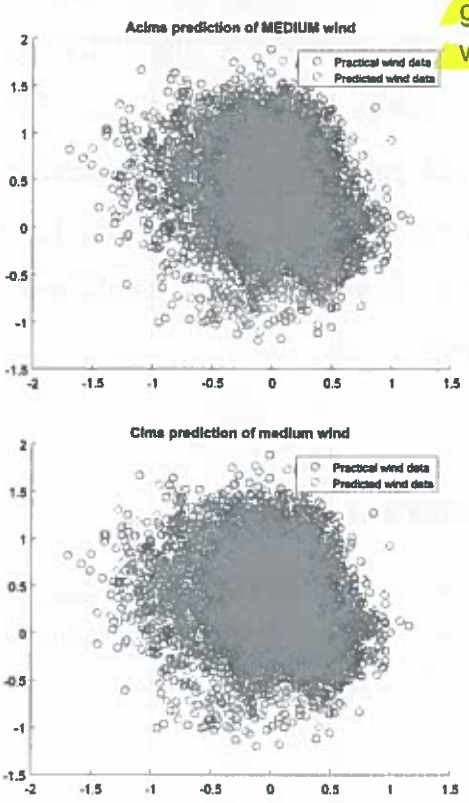
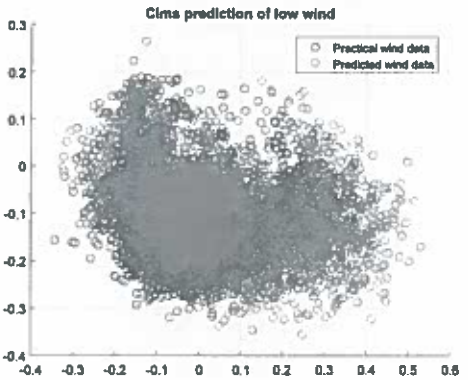


Figure 71: ACLMS and CLMS estimation in mid wind



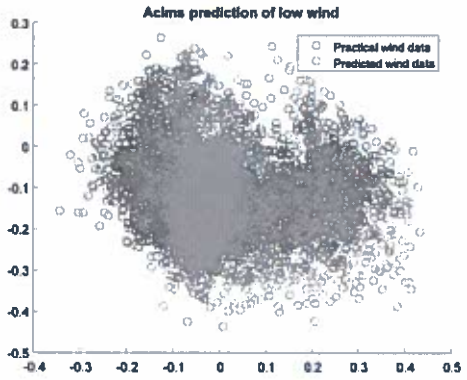


Figure 72: ACLMS and CLMS estimation in low wind

And the report is not complete.

It is found that for signal has good circularity, both ACLMS and CLMS could provide satisfied prediction of the signal. However, for signal has not ideal circularity, CLMS could provide better results.

References

[1] D.Mandic, "Adaptive signal processing & machine intelligence lecture 3," London, 2017.